

Conception Architecturale

Rapport de conception d'un filtre numérique à 4 coefficients

Table des matières

1.	Introduction.....	3
2.	Conception d'une architecture parallèle pour l'implémentation du filtre	3
3.	Filtre séquentiel.....	12
4.	Analyse du temps pour la solution séquentielle	16
5.	Comparaison.....	16
6.	Conclusion.....	17

1. Introduction

Ce travail pratique porte sur l'étude et la mise en œuvre d'un filtre numérique à réponse impulsionnelle finie RIF. À partir de l'équation du filtre et de son graphe flot de données, plusieurs architectures matérielles sont analysées, décrites en VHDL. Les approches parallèle, séquentielle seront comparées selon différents critères : latence, fréquence maximale, débit, ainsi que coût en ressources matérielles.

L'objectif de ce TP est de comprendre l'impact de l'ordonnancement des opérations et des choix architecturaux sur les performances d'un système numérique, tout en confrontant les estimations théoriques aux résultats obtenus après synthèse.

Le filtre numérique que nous étudierons prendra la forme mathématique suivante :

$$y(n) = \frac{1}{16} (7x(n) + 5x(n-1) + 3x(n-2) + x(n-3))$$

Nous implémenterons la solution sur la carte FPGA MAX 10.

2. Conception d'une architecture parallèle pour l'implémentation du filtre

Nous réalisons ici dans un premier temps le graphe flot de donnée de l'architecture parallèle.

De la description fonctionnelle du filtre (l'équation du filtre), nous identifions les entrées/sorties de données, les opérateurs de calcul nécessaires et la dépendance temporelle entre les opérations. Nous obtenons après analyse, le graphe flot de données suivant :

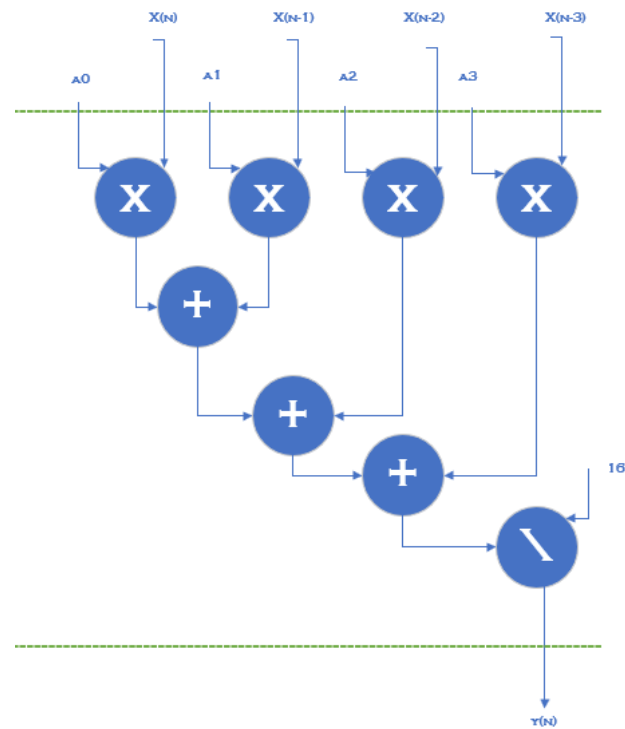
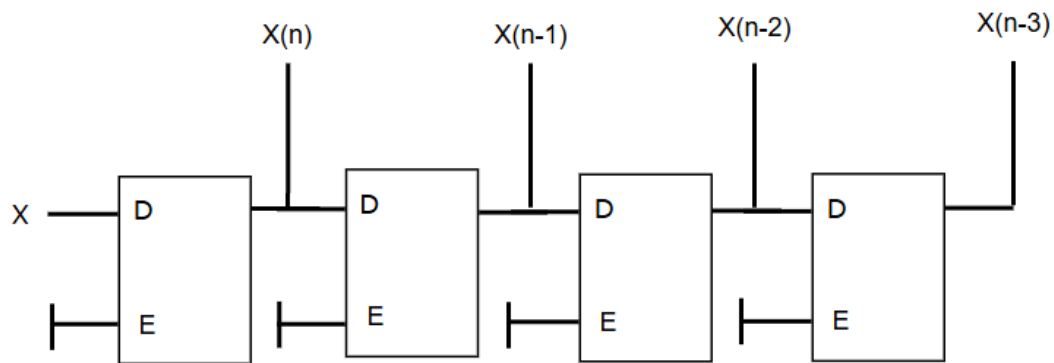


Figure 1 : DFG (Graphe Flot de Donnée) de l'architecture parallèle

Les entrées $x(n)$, $x(n-1)$, $x(n-2)$, $x(n-3)$ sont obtenus à l'aide de l'architecture simplifiée suivante :



L'horloge est la même pour toutes les bascules et le signal E est à 1 en permanence.

Figure 2 : Circuit d'acquisition des échantillons en entrée

Commentaires sur la solution proposée :

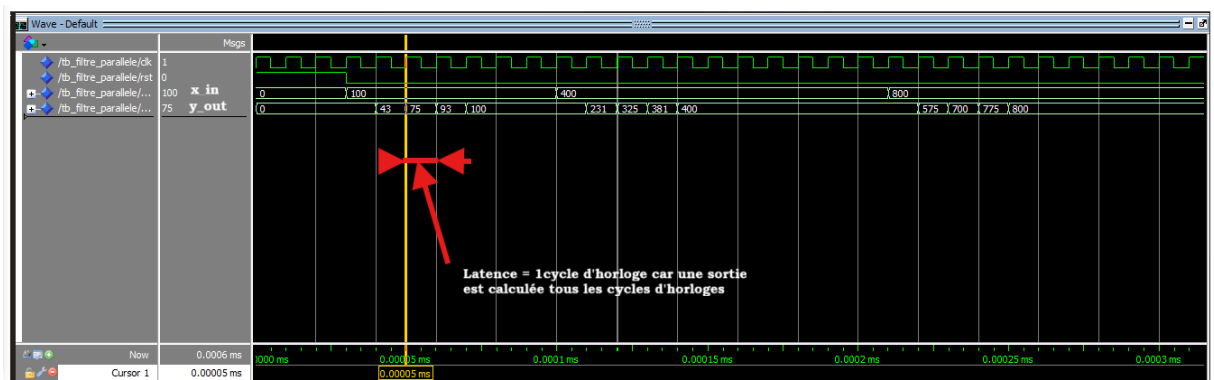
- Identification des ressources nécessaires et coût
 - Sept (7) opérateurs sont nécessaires dont quatre (4) multiplieurs et trois (3) additionneurs. La solution étant parallèle, nous nous devons de nous assurer que les ressources sont toutes disponibles au même moment.

En termes de ressources matérielles nécessaires, nous aurons donc besoin de quatre (4) multiplieurs et trois (3) additionneurs. Le diviseur par 16 ne prendra aucune ressource car il s'agit juste de sélectionner les bits significatifs qu'à partir du bit 4. Les autres bits sont reliés à une masse.

Coût = 4 Multiplieurs + 3 Additionneurs

- La Latence

La latence est le temps qui s'écoule entre l'entrée de donnée $X(n)$ et la sortie après traitement. Du graphe flot de données on peut tout de suite déduire, que le calcul prendra un (1) cycle. Nous avons vérifié ce résultat par la simulation de la solution parallèle décrite en VHDL (langage de description matérielle).



- Le temps du chemin critique

$T_{cc} = 1 \times \text{Temps de multiplieur} + 3 \times \text{Temps de l'additionneur} + \text{Temps de registre}$

- La fréquence maximale de fonctionnement

$$F_{Hmax} = \frac{1}{(T_{cc} + t_{su} + t_{skew} + t_{jitter} + \text{marge})}$$

- Le débit

Le débit étant la vitesse maximale de traitement des données en entrée ou en sortie, elle est de F_{Hmax} .

Etude de l'utilisation des DSP dans la solution parallèle

Inférer des blocks DSP embarqués dans une carte FPGA peut être utile pour améliorer les performances d'un circuit dans certains cas. Nous allons explorer ici, l'impact de l'utilisation des blocs DSP.

Au cours de notre phase de conception, nous avons remarqué que malgré la configuration du logiciel Quartus dans le but que les blocs DSP soient utilisés automatiquement, comme cela se fait d'habitude, ceux-ci ne sont pourtant jamais utilisés.

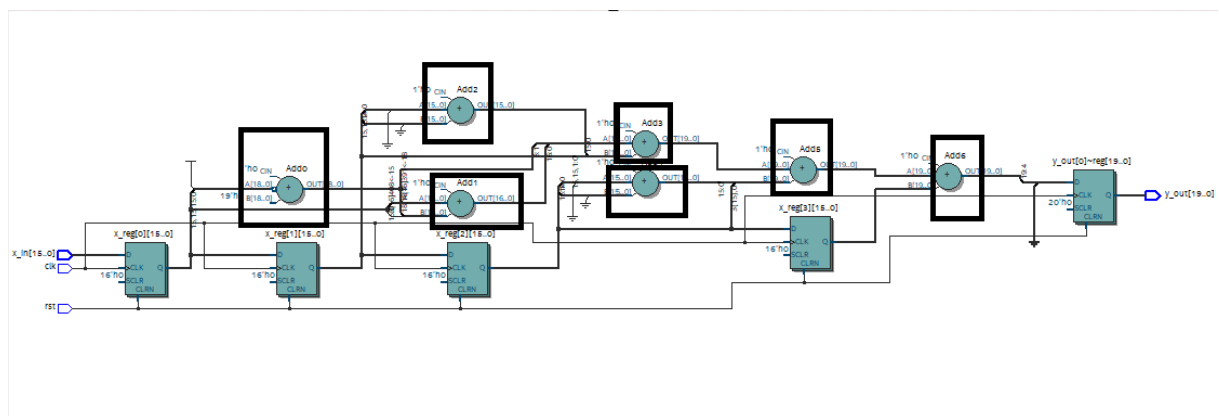
Name:	Setting:
Auto DSP Block Replacement	On
DSP Block Balancing	DSP blocks
Maximum DSP Block Usage	-1 (UNLIMITED)

Nous estimons après notre recherche bibliographique que la cause nous vient du fait que nous avons déclaré les coefficients de notre filtre comme des constantes. Nous supposons que dans une multiplication, lorsque des constantes sont déclarées avec le mot clé « constant » dans VHDL, le logiciel Quartus n'utilise pas les DSP pour les produits partiels. Les blocs DSP étant limités, ils sont économisés car le logiciel Quartus considère que faire les opérations en utilisant les LUT(Look Up Tables) et le décalage est plus performante.

Par exemple lorsqu'on à faire par exemple le produit de $9 \cdot x$, on peut faire :

$9 \cdot x = (8 \cdot x + x) = (x \ll 3) + 1$. Cette opération peut se faire entièrement en combinatoire en utilisant un simple additionneur et en faisant des opérations de décalage qui ne coûtent pas en termes de ressources et de performance.

Nous pouvons confirmer notre hypothèse en observant le schéma RTL de la synthèse qui nous dit montre l'utilisation de 7 blocs additionneurs alors seulement 3 opérations d'addition sont nécessaires avec notre filtre. Les 4 autres additionneurs utilisés ont certainement servi à faire les 4 opérations de multiplication nécessaires dans le calcul de notre filtre.

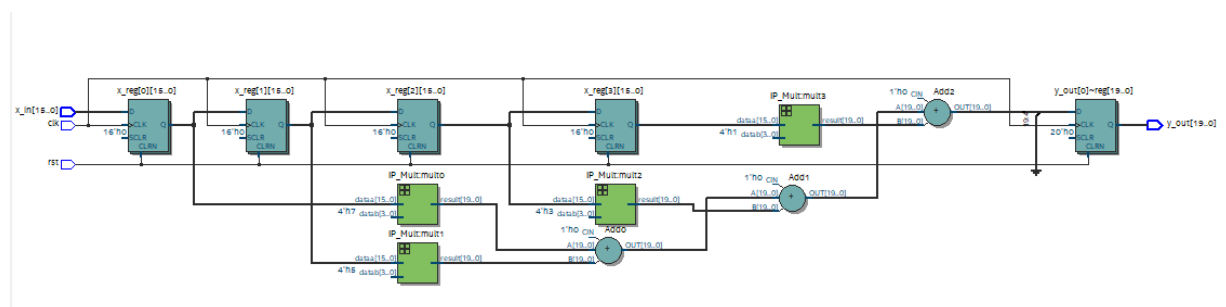


Flow Summary

<<Filter>>

Flow Status	Successful - Sat Nov 15 11:06:48 2025
Quartus Prime Version	16.1.0 Build 196 10/24/2016 SJ Lite Edition
Revision Name	filtre_parallele
Top-level Entity Name	filtre_parallele
Family	MAX 10
Device	10M08DAF484C8G
Timing Models	Final
Total logic elements	144 / 8,064 (2 %)
Total registers	80
Total pins	38 / 250 (15 %)
Total virtual pins	0
Total memory bits	0 / 387,072 (0 %)
Embedded Multiplier 9-bit elements	0 / 48 (0 %)
Total PLLs	0 / 2 (0 %)
UFM blocks	0 / 1 (0 %)
ADC blocks	0 / 1 (0 %)

Nous pouvons malgré le refus de Quartus d'utiliser des blocs DSP, utiliser des blocs IP de multipliers fournis par Quartus pour la carte FPGA en question (MAX10), un moyen excellent d'utiliser les blocs DSP selon Quartus.



Nous pouvons à partir du schéma RTL confirmer notre hypothèse selon laquelle Quartus utilise l'addition et le décalage pour les opérations de multiplication dans le cas d'une multiplication par une constante.

Flow Summary	
<<Filter>>	
Flow Status	Successful - Sat Nov 15 10:47:35 2025
Quartus Prime Version	16.1.0 Build 196 10/24/2016 SJ Lite Edition
Revision Name	filtre_parallele
Top-level Entity Name	filtre_parallele
Family	MAX 10
Device	10M08DAF484C8G
Timing Models	Final
Total logic elements	104 / 8,064 (1 %)
Total registers	64
Total pins	38 / 250 (15 %)
Total virtual pins	0
Total memory bits	0 / 387,072 (0 %)
Embedded Multiplier 9-bit elements	8 / 48 (17 %)
Total PLLs	0 / 2 (0 %)
UFM blocks	0 / 1 (0 %)
ADC blocks	0 / 1 (0 %)

Dans le cas où nous déclarons les constantes comme des vecteurs d'entrée, donc sans utiliser le mot clé constante, nous remarquons que les blocs DSP sont utilisés lorsque le logiciel Quartus est paramétré pour :

Flow Summary	
 <<Filter>>	
Flow Status	Successful - Sat Nov 15 12:03:34 2025
Quartus Prime Version	16.1.0 Build 196 10/24/2016 SJ Lite Edition
Revision Name	filtre_parallele
Top-level Entity Name	filtre_parallele
Family	MAX 10
Device	10M08DAF484C8G
Timing Models	Final
Total logic elements	104 / 8,064 (1 %)
Total registers	68
Total pins	54 / 250 (22 %)
Total virtual pins	0
Total memory bits	0 / 387,072 (0 %)
Embedded Multiplier 9-bit elements	8 / 48 (17 %)
Total PLLs	0 / 2 (0 %)
UFM blocks	0 / 1 (0 %)
ADC blocks	0 / 1 (0 %)

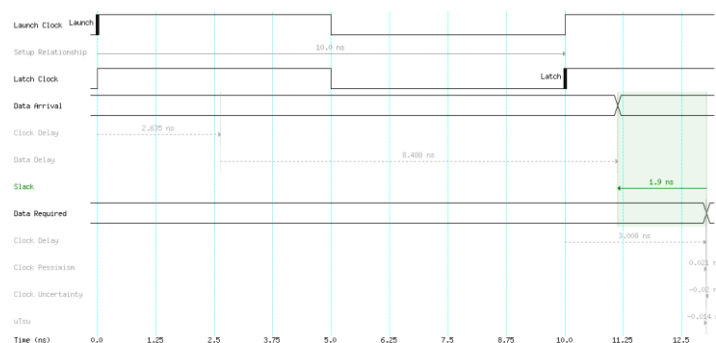
8 blocs DSP utilisés.

Analyse du timing pour la solution parallèle

Nous pourrions nous intéresser à comparer diverses situations :

Les différentes situations abordées sont les suivant :

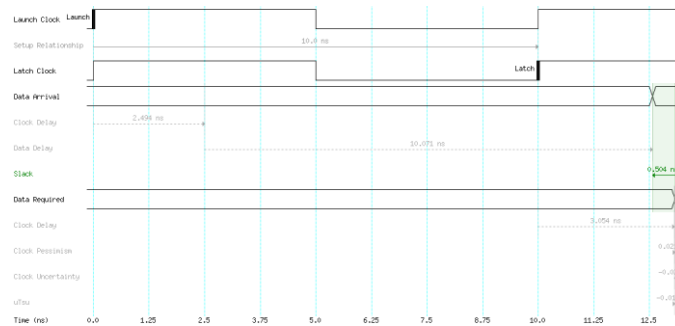
- Les coefficients sont déclarés comme des constantes dans la description VHDL avec le mot clé <<constant>>
 - Sans utiliser les blocs DSP



$F_H \approx 1/T_{cc} \approx 1/8.488\text{ns} \approx 117,81\text{ MHz}$, ou 8.488ns représente le Data Delay

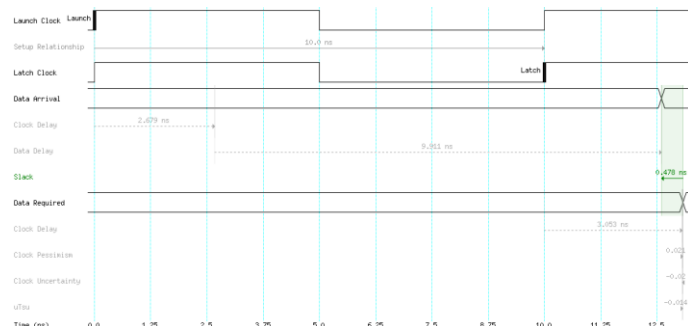
- En forçant l'utilisation des blocs DSP par instantiation d'un bloc IP de multiplieur

Note : Dans cette situation, même si Quartus est paramétré pour ne pas utiliser les blocs DSP, ils sont quand même utilisés



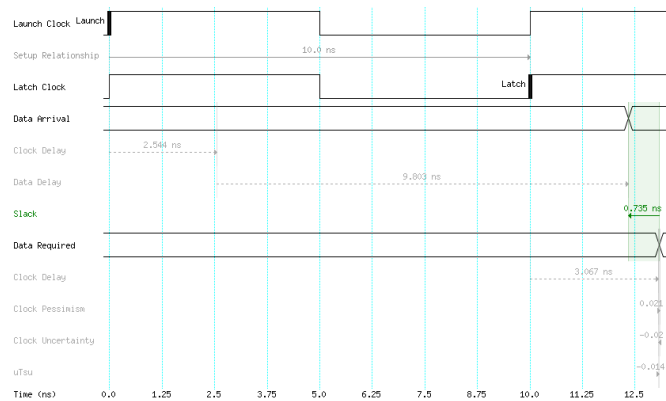
$$F_H \approx 1/T_{cc} \approx 1/10.071\text{ns} \approx 99,295\text{ MHz}, \text{ ou } 10.071\text{ns} \text{ représente le Data Delay}$$

- Les coefficients sont déclarés comme des vecteurs d'entrée sans utiliser le mot clé <<constant>>
 - Sans utiliser les blocs DSP



$$F_H \approx 1/T_{cc} \approx 1/9.911\text{ns} \approx 100,91\text{ MHz}, \text{ ou } 9.911 \text{ représente le Data Delay}$$

- En utilisant les blocs DSP



$$F_H \approx 1/T_{cc} \approx 1/9.803\text{ns} \approx 102\text{ MHz} \text{ ou } 9.803 \text{ représente le Data Delay}$$

Figure 3 : RTL parallèle issue de Quartus

Tableau comparatif des solution parallèles

- Sans DSP

Slow 1200 mV 85C model	Type coefficients = constantes	Type coefficients=signaux
Latence	1	1
Registres	80	80
Élément logique	144	440
Temps chemin critique(ns)	8.488	9.911
F _H max(MHz)	117.81	100.91
Débit	F _H max	F _H max

- Avec DSP

Slow 1200 mV 85C model	Type coefficients = constantes	Type coefficients=signaux
Latence	1	1
Registres	64	68
Élément logique	104	104
DSP	8	8
Temps chemin critique(ns)	10.0711	9.803
F _H max(MHz)	99.795	102
Débit	F _H max	F _H max

Des deux tableaux on peut remarquer que la meilleure solution en termes de performances est celle où les coefficients sont gravés en dur sur la carte FPGA, donc déclarés comme constants. Le chemin critique est le plus court.

La solution avec les coefficients déclarés comme signaux est très gourmande en ressources comparativement aux autres. Elle prend 4 fois plus de portes logiques que les autres mais ne donne pas les meilleures performances.

Dépendamment du compromis la surface/performance souhaité, un des autres choix serait le plus favorable.

Forcer l'utilisation des blocs DSP, à travers l'utilisation des blocs IP à permis ici de réduire les ressources utilisées. Toutefois, les performances se sont dégradés.

3. Filtre séquentiel

Nous réaliserons le graphe flot de donnée de la solution séquentielle. Il existe en effet plusieurs possibilités pour l'ordonnancement des opérations dans la solution séquentielle. Nous voulons trouver un bon compromis entre ressources matérielles (espace) et performances (temps) du circuit.

Pour ce faire nous avons repéré une série d'opérations qui se répète plusieurs fois dans l'algorithme de la fonction à réaliser. Les opérateurs de cette série d'opération peuvent être réutilisés dans plusieurs cycles.

Le graphe flot de données obtenu se présente comme suit :

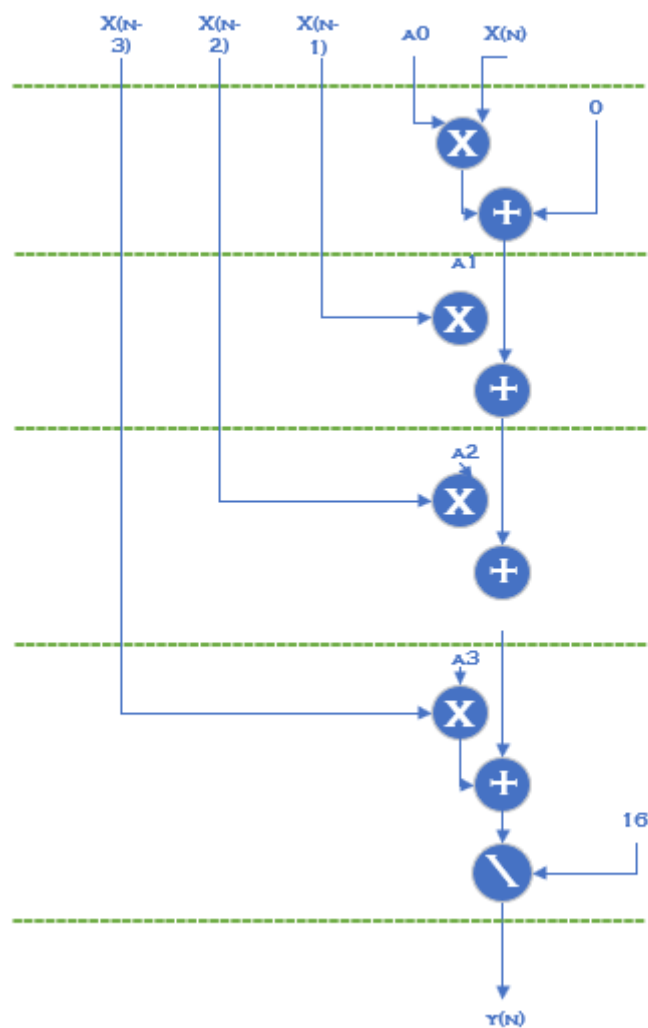


Figure 4 : Figure 5 : DFG (Graphe Flot de Donnée) de l'architecture séquentielle

- Identification des ressources nécessaires et coût
 - De 7 opérateurs pour la solution parallèle, nous sommes passés à 2 opérateurs nécessaires pour la solution séquentielle. Nous aurons besoin d'un (1) multiplieur et d'un (1) additionneur. Tout comme précédemment, le diviseur par 16 ne prendra aucune ressource car il s'agit juste de sélectionner les bits significatifs qu'à partir du bit 4. Les autres bits sont reliés à une masse.

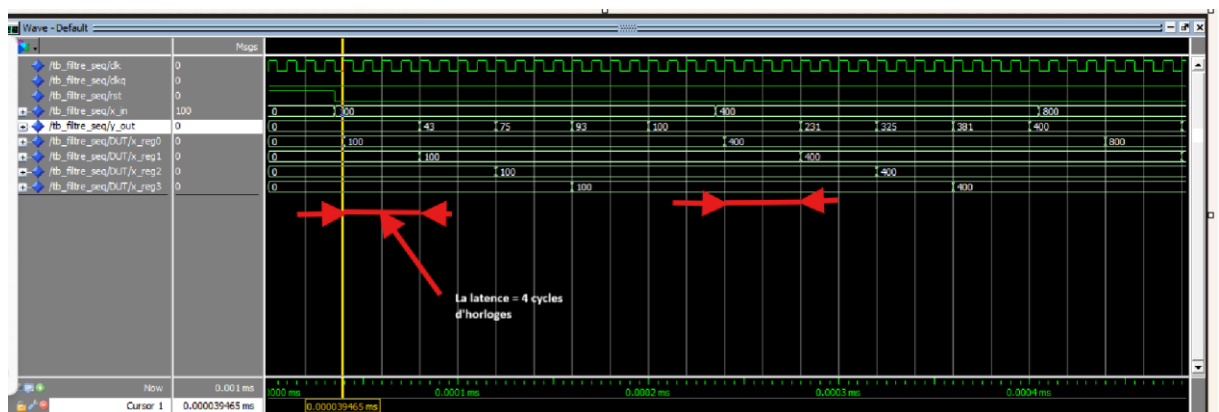
$$\text{Coût} = 1 \text{ Multiplieur} + 1 \text{ Additionneur}$$

- La Latence

La latence est le temps qui s'écoule entre l'entrée de donnée $X(n)$ et la sortie après traitement.

Du graphe flot de données on conclut que la latence est de 4 cycles.

Nous avons vérifié ce résultat par la simulation de la solution séquentielle décrite en VHDL (langage de description matérielle).



- Le temps du chemin critique

$T_{cc} = 1 \times \text{Temps de multiplieur} + 1 \times \text{Temps de l'additionneur} + \text{Temps de registre}$

- La fréquence maximale de fonctionnement

$$F_{Hmax} = \frac{1}{(T_{cc} + t_{su} + t_{skew} + t_{jitter} + \text{marge})}$$

- Le débit

Le débit étant la vitesse maximale de traitement des données en entrée ou en sortie, elle est de :

$$D = \frac{F_{Hmax}}{4}$$

Machine à état de la solution :

Une machine à état de 4 cycles nous permet de traverser les 4 cycles de notre DFG(Graphe Flot de Données). Elle est présentée comme suit :

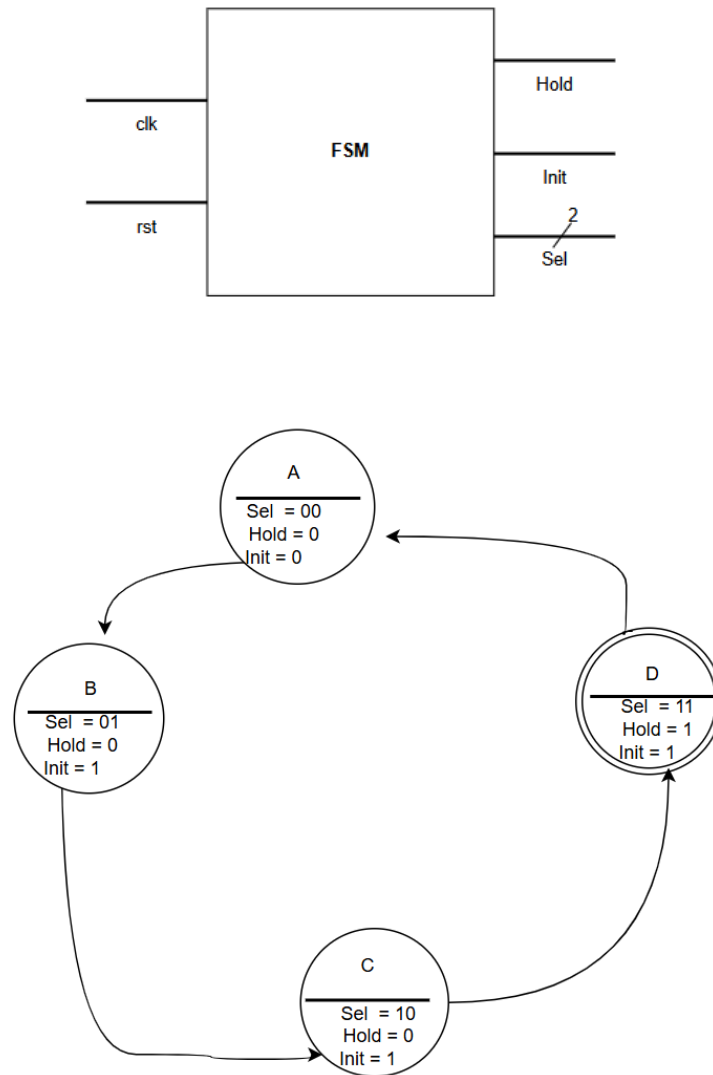


Figure 6 : Machine d'état

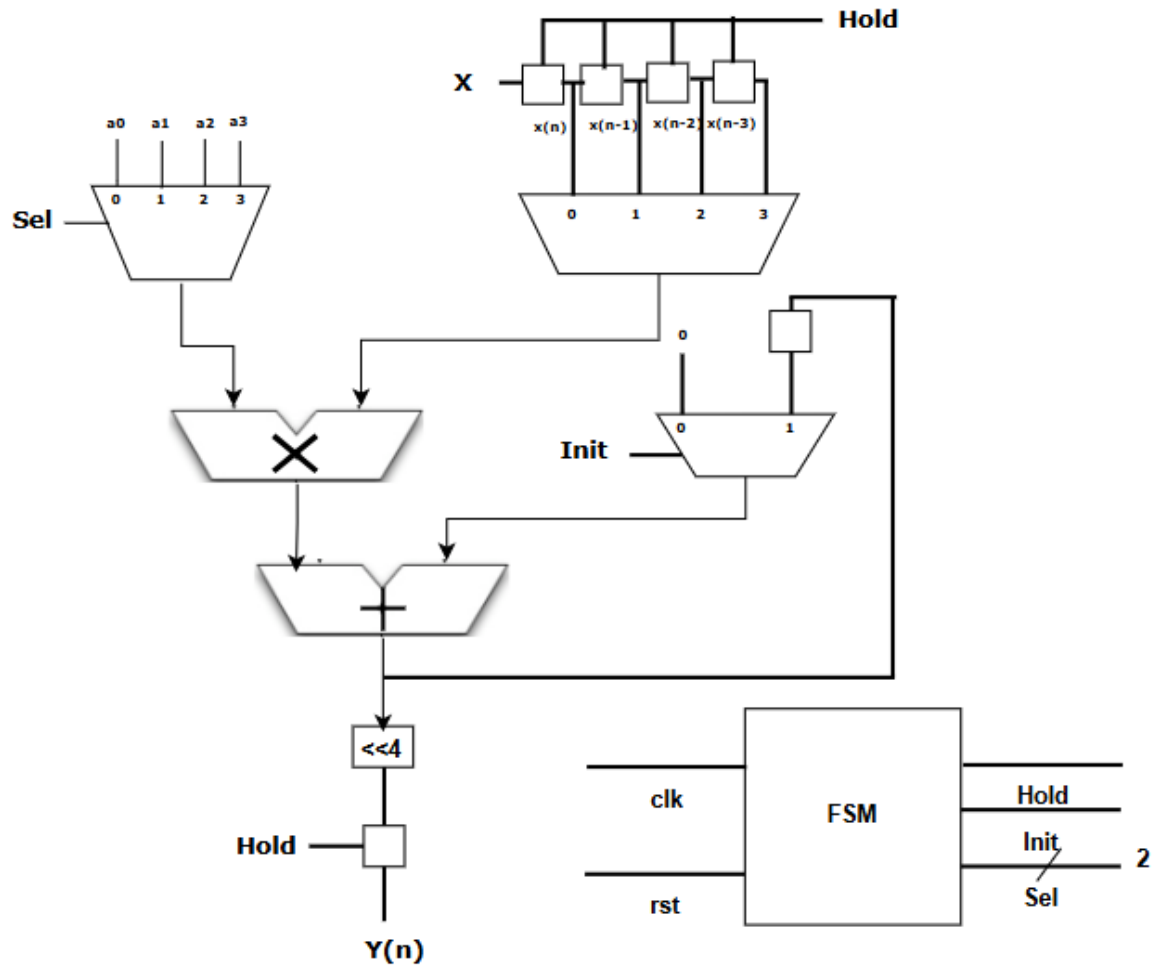


Figure 7 : Architecture de la solution séquentielle

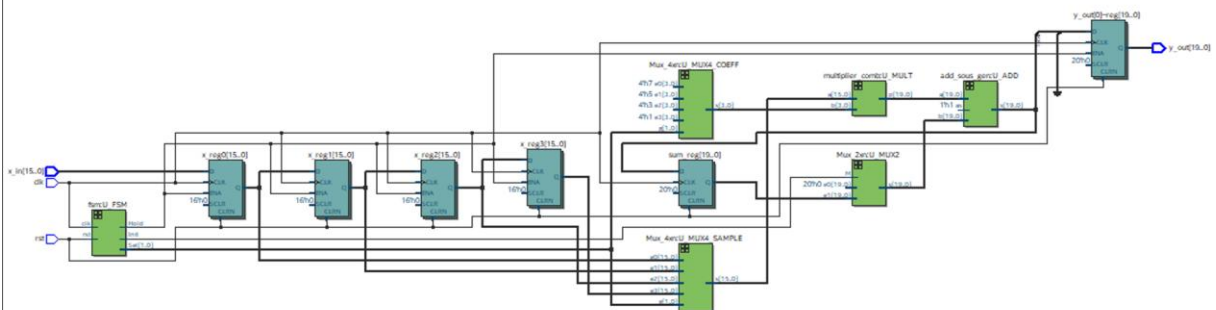
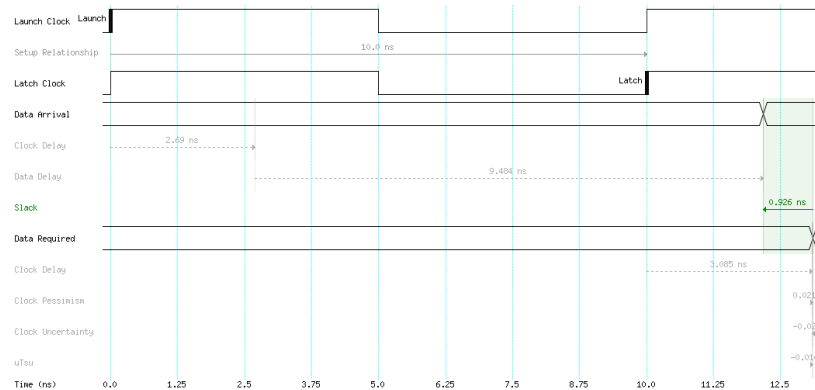


Figure 8 : RTL de la solution séquentielle issue de quartus

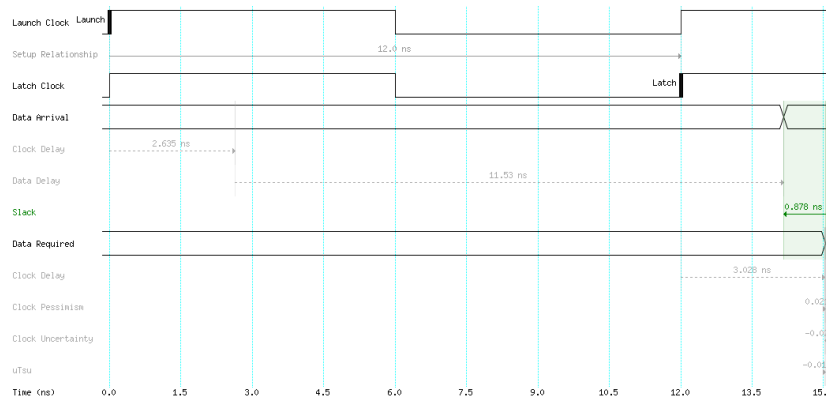
Analyse du temps pour la solution séquentielle

Sans block DSP



$$F_H \approx 1/T_{cc} \approx 1/9.484\text{ns} = 105,84 \text{ MHz ou } 9.48 \text{ représente le Data Delay}$$

Avec block DSP



$$F_H \approx 1/T_{cc} \approx 1/11.53\text{ns} \approx 86,7 \text{ MHz ou } 9.369 \text{ représente le Data Delay}$$

4. Comparaison

Note : L'architecture parallèle présentée dans le tableau de comparaison est celle faite avec les coefficients des filtres déclarées comme des signaux

Sans bloc DSP

Slow 1200 mV 85C model	Parallèle	Séquentiel
Latence	1	4
Registres	80	104
Élément logique	440	188
Temps chemin critique(ns)	9.911	9.484
$F_{H\text{max}}$ (MHz)	100.91	105.84
Débit	$F_{H\text{max}}$	$F_{H\text{max}}/4$

Avec bloc DSP

Slow 1200 mV 85C model	Parallèle	Séquentiel
Latence	1	4
Registres	68	104
Élément logique	104	188
DSP	8 multiplieurs 9 bits	2 multiplieurs 9bits
Temps chemin critique(ns)	9.803	9.369
F_{Hmax} (MHz)	102	86,7.
Débit	F_{Hmax}	$F_{Hmax}/4$

5. Conclusion

Pour ce filtre numérique, la meilleure solution serait d'utiliser la solution parallèle sans les blocks DSP avec les coefficients gravés comme constants, si l'on n'a pas besoin de changer les coefficients après.

Elle fournit non seulement les meilleures performances mais aussi un bon compromis en terme de surface.

Utiliser les blocs DSP peut parfois rendre une solution meilleure en termes de performances ou non. Pour le filtre séquentiel par exemple les performances se sont dégradées avec l'utilisation des blocs DSP tandis que la surface occupée par le circuit ne s'est pas pour autant réduite.

